

Deltazium

Debezium + Kafka 기반 Oracle CDC 파이프라인 · 복구(replay) 아키텍처 · 웹 제어 콘솔

최남희 · 단독 설계·개발 · 2026.07-08 · github.com/namhee0812/Deltazium · 상용 CDC 솔루션(DeltaStream) 개발 경험 기반의 검증 프로젝트

1. 프로젝트 개요

Oracle의 변경(CDC)을 LogMiner로 캡처해 **같은 스트림을 두 갈래로 병행 적재**한다 — 타깃 Oracle에는 현재 상태를(실적재), Iceberg/S3에는 모든 변경 이력을(append-only changelog). changelog는 미러링이 아니라 **특정 SCN 시점부터의 복구(replay)를 위한 원본**이며, 이 복구 아키텍처의 성립을 실제 배선으로 증명하는 것이 프로젝트의 목적이다. 그 위에 테이블 등록·사전 점검·DDL 승인·복구를 담당하는 제어면(Spring Boot + React)을 얹었고, 운영 중 실제 발생한 장애(archive log 소실로 인한 캡처 정지)를 계기로 **장애 감지 → 원인 표시 → 버튼 한 번의 복구(단계형 재스냅샷)**까지 운영 사이클을 완성했다. 시계열 모니터링(처리량·lag·자원)은 자체 경량 수집·2단 롤업으로 구현했다.

93

자동화 테스트

11.2k

코드 라인 (Java 7.4k · TS 3.8k)

53만+

실트랙픽 CDC 이벤트

2건

실장애 복구 (유실 0 · 완전 재구축)

2. 아키텍처

```
Oracle(SRC) —Debezium source(LogMiner)—> Kafka(KRaft) —┐ Debezium JDBC sink —> Oracle(TGT) · 실적재(현
└─ Iceberg sink (append) —> Iceberg / MinIO(S3)

복구: recovery-job —(Iceberg scan · SCN 순서 복원 · envelope 재조립)—> 복구 토픽 —> recovery-sink(동일 apply
[Web UI(React)] ⇄ [backend(Spring Boot): 등록·사전 점검 · DDL 승인 · 복구 트리거 · 이벤트 이력 · 실측 메트릭]
```

컴포넌트	역할	보존/특성
Kafka (KRaft)	짧은 버퍼 · fan-out (두 sink가 독립 consumer group — lag 격리)	retention 24h — 이력 보관 책임 없음
Iceberg / MinIO	장기 이력 · 복구 원본 (envelope-as-is, append-only)	1일 파티션 drop = 보존 정책
Oracle 타깃	현재 상태 (PK upsert 먹등 apply)	—

3. 핵심 설계 판단

먹등 apply를 전제로 한 복구 단순화

전달 보장은 at-least-once로 두고, apply를 PK 기반 upsert(MERGE)로 통일해 **몇 번을 재적용해도 결과가 같게** 만들었다. 복구 재생의 경계 중복을 정밀 절단 없이 흡수하는 근거이며, PK 없는 테이블은 등록 단계에서 거부한다. 복구는 재발행 단일 경로 — recovery-job은 Iceberg를 읽어 복구 토픽에 재발행까지만 하고, apply는 라이브와 동일한 JDBC sink 설정이 담당해 **복구 결과와 라이브 적재가 달라질 수 없는 구조**다.

changelog 스키마를 실측 실험으로 확정

원안(평탄화 스키마)이 기성 커넥터·SMT 조합으로 구현 불가함을 실험으로 증명하고 (스톡 SMT는 중첩 필드 승격 불가, 기존 변환 SMT는 before 이미지 소실 — 무손실 재조립 불변식 위반), **envelope 구조 그대로 저장 + backend가 테이블 사전 생성 + 1일 파티션**으로 설계를 개정했다. envelope 왕복 테스트(원본 → Iceberg 행 → 재조립 동등성)가 이 불변식의 회귀 방어선이다.

캡처 장애의 재현·진단·전환 — 유실 0

공유 dev Oracle에서 LogMiner redo_log_catalog 전략이 ORA-1371(complete dictionary not found) 재시도 루프에 빠져 커넥터는 **RUNNING**인데 **캡처가 멈추는** 장애를 재현했다. 디렉터리 아카이브 존재를 소스 디렉터리 조회로 확인해 전략 자체의 취약점으로 진단, online_catalog 로 전환(DDL 이벤트는 양 전략 모두 발행됨을 확인해 승인 워크플로 유지)했고, 소스 redo 재마이닝으로 밀린 9,673건을 전량 회수했다. 커넥터 삭제 후 재등록 시 잔존 offset으로 스냅샷이 건너뛰어지는 함정도 실측으로 발견해 등록 해제 시 offset 정리를 자동화했다.

캡처 계정 최소 권한 8개 도출

Debezium 문서의 보수적 권한 목록(20여 개)을 실 배선의 권한 실패를 재현하며 8개로 압축. `SELECT ANY DICTIONARY` 가 V\$ 뷰 개별 grant를 대체함을 확인하고, `DBMS_LOGMNR_D` (디렉터리 전략 전용)·`FLASHBACK ANY TABLE` (초기 스냅샷 전용 — flashback query 일관성)의 용도를 구분해 문서화했다. 사전 점검이 권한 누락 시 DBA용 GRANT 스크립트를 생성해 안내하고 배포를 차단한다.

운영 동작의 원칙화

jdbc-sink를 테이블별 커넥터로 분리해 타깃 이름 매핑과 테이블 단위 정지(DDL 거부 격리)를 가능하게 했고, recovery-sink는 apply 완료(consumer lag 소진)를 감지해 자동 정지한다("평시 정지" 원칙). 복구 트리거의 자동 재개 옵션으로 **"정지 → S3 복구 → 라이브 복귀(go-live)"**가 **한 번의 조작으로 완결**된다 — 경계 중복은 전부 멍등이 흡수하므로 절단 시점 계산이 필요 없다.

실장애: connector RUNNING / task FAILED — 나홀 미감지의 해부와 복구

공유 dev Oracle의 archive log 정리로 재개 지점 SCN이 소실돼 캡처 task가 죽었는데, connector는 RUNNING·발행이 없으니 lag도 0 — **모든 지표가 "평온"을 가리키는 나홀**이 흘렀다. Connect REST의 task trace로 원인을 확정하고, 이 장애에서 기능을 파생시켰다: task 포함 상태 판정(최악값 집계), 장애 배너(감지 시각·원인 한 줄·전체 trace), 행별 재시도, 그리고 **단계형 재스냅샷 워크플로**. 최종 복구는 타깃 truncate 완전 재구축으로 10.5만 행을 50초에 go-live — 갭 중 소스에서 삭제된 행(고아 +2,237)까지 정리했다. 전 과정을 장애 기록 문서(타임라인·진단 경로·교훈)로 남겼다.

단계형 재스냅샷 — 승인·홀드가 있는 상태 기계

재스냅샷을 블랙박스 버튼이 아니라 **관찰 가능한 단계**로 만들었다: 유입 차단 → 파이프 잔량 소진(실시간 lag) → 타깃 비우기 승인(시스템 실행은 권한 사전 점검, 권한 불충분·직접 실행은 **DBA 홀드** — 테이블별 count=0을 폴링해 자동 진행) → 재기동 → 스냅샷 → go-live. 되돌릴 수 없는 지점(offset 삭제) 전까지는 취소·실패 시 원복이 보장되고, 스냅샷 진행률은 Debezium notification 채널을 소비해 **추정이 아닌 실측**으로 표시한다 (테이블별 스캔 행수·완료 전 이).

모니터링: 자체 경량 수집 → 2단 롤업

1분 샘플러(토픽 offset 델타·sink lag·pid 파일 기반 /proc CPU·RSS)를 PG에 적재하고, 완결 구간만 대상으로 **분(48h) → 시간(60일) → 일(1년)** 롤업한다 — NOT EXISTS 먹등이라 재실행 중복이 없고, 집계는 지표 성격대로(처리량=합, lag=구간 최대로 스파이크 보존). 대시보드의 **발행 vs apply 겹침 차트**는 두 선이 벌어지는 순간이 곧 lag 누적이라는 조기 신호로, "lag 0은 따라잡음과 무소식을 구분 못 한다"는 이번 장애의 교훈을 화면으로 만든 것이다. Prometheus/Grafana는 의도적으로 미도입(백로그) — 무엇을 보여줘야 하는가를 먼저 검증했다.

4. 웹 콘솔 주요 기능

화면	기능
대시보드	토폴로지(자체 SVG — 교차 없는 직교 라우팅·흐름 애니메이션) + 처리량(발행 vs apply)·lag 시계열(테이블 검색 선택·최근 6시간/7일/90일) + 컴포넌트 자원(/proc 실측) + 최근 운영 이벤트
CDC 등록 위저드	6단계: 소스 선택 → 디렉터리 조회(와일드카드 전개, PK 없는 테이블 선택 차단) → 타깃 → 컬럼 매핑(체크박스 제외· <code>\${소스컬럼}</code> 변환식·구문 검증) → 사전 점검(supp.log 승인 후 적용·권한 GRANT 안내) → 스냅샷 모드 선택(초기적재/현재부터)·배포
테이블 모니터링	실측 지표 — 총 이벤트(offset)·이벤트/s·sink별 lag(AdminClient), 정지/재개/재시도/삭제, 캡처 장애 배너(원인·trace·복구 진입), 재스냅샷(단계 팝업·truncate 재구축)
DDL 이력	schema change topic 상시 수집(retention 만료 대비 DB 적재), 승인(타깃 이름 치환 실행)/거부(테이블 단위 apply 정지), 비전파성 DDL 자동 무시
이벤트	테이블별 운영 이력 — 등록·정지·DDL·복구·커넥터 장애 전이(원인 trace 포함) 타임라인
복구	changelog(S3) 현황(용량·보존 구간·파티션별)·SCN 지정 재발행·go-live 자동 재개·SRC/TGT 정합 검증(행수+체크섬)

5. 검증 결과

- ✓ 실 Oracle end-to-end: 초기 스냅샷(30,088행) → LogMiner 실시간 스트리밍 → 타깃 apply(lag 0) + changelog 커밋 까지 실트래픽 확인
- ✓ 캡처 장애(ORA-1371) 재현 → 전략 전환 → 밀린 9,673건 유실 0 회수
- ✓ envelope 왕복 동등성·changelog 사전 생성(실 카탈로그)·복구 재발행 스모크(73건) 등 자동화 테스트 72건
- ✓ 사전 점검 게이트 실동작: LogMiner 권한 누락 감지 → GRANT 안내 → 부여 후 통과
- ✓ 실장애 복구: archive log 소실로 죽은 캡처(나홀 경과)를 재스냅샷 워크플로로 복구 — truncate 완전 재구축 10.5만 행·50초 go-live, 스냅샷 진행률 notification 실측
- ✓ 정합 검증(행 수+ORA_HASH 체크섬)으로 고아 행 편차(+2,237) 검출 → 재구축 후 수렴 확인
- ✓ 모니터링 실측: 분당 2,262건 버스트에서 발행·apply 추적, lag 스파이크·소진 시계열 기록, 롤업 먹등 테스트

6. 기술 스택

Java 21	Spring Boot 3.5	MyBatis	Debezium 3.6 (Oracle LogMiner · JDBC sink)	Kafka 4.3 KRaft	Apache Iceberg 1.11
MinIO(S3)	PostgreSQL 15	React 19	TypeScript	Vite	Tailwind 4 · shadcn/ui
자체 SVG (토폴로지·차트)					TanStack Table

데이터 경로는 기성 커넥터만 사용(자체 코드는 제어면·복구·UI에 한정 — apply 시맨틱 단일 경로 유지가 설계 원칙). 인프라는 베어메탈 스크립트로 먹등 구축(Iceberg Kafka Connect sink는 공식 프리빌드가 없어 소스 빌드). Iceberg 읽기는 Spark/Trino 없이 iceberg-data 라이브러리 단일 프로세스.

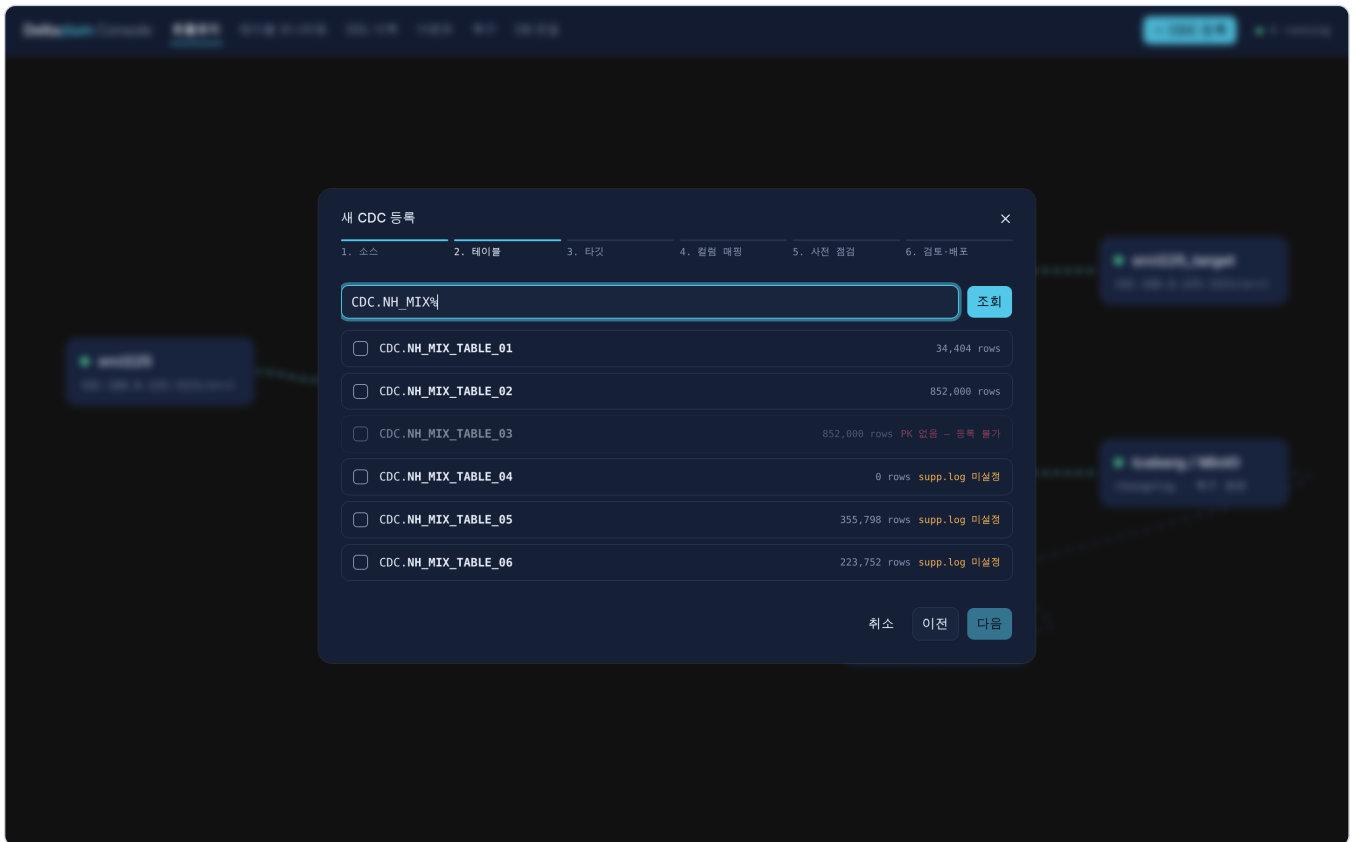
7. 문서화 · 남은 과제

설계 기준(architecture.md)·운영 가이드(operations.md)·구현 내부 노트(internals.md)·실장애 기록(incidents/)·실측 실험 기록(experiments/)을 리포에 유지 — "UI에는 사용자 결정에 필요한 것만, 시스템 내부는 문서·주석에" 원칙. 남은 과제: 기동 중 테이블 추가 시 초기적재(incremental snapshot + Kafka signal) · Prometheus/Grafana(JMX exporter) 도입 후 자체 수집과 비교 · 컬럼 리네임의 적재 반영(스톡 sink 한계 — 정책 결정 대기) · S3 복구 풀 리허설 실행 검증

8. 콘솔 화면



대시보드 — 토폴로지(자체 SVG)·처리량(발행 vs apply)·lag 시계열·컴포넌트 자원·최근 이벤트 (분당 2,262건 버스트 실측)



등록 위저드 — 소스 디렉터리 조회: PK 없는 테이블 선택 차단·supplemental logging 경고·행수

Deltazium Console

대시보드테이블 모니터링DDL 이력이벤트복구DB 연결

+ CDC 등록

● 커넥터 4 running

스키마,테이블 검색

새로고침재스냅샷

2 tables

TABLE	TOPIC	EVENTS	EVENTS/s (실속)	JDBC LAG	ICEBERG LAG	
● CDC.NH_MIX_TABLE_01	dz.CDC.NH_MIX_TABLE_01	358,966	0.0/s	0	0	정지삭제
● CDC.NH_MIX_TABLE_02	dz.CDC.NH_MIX_TABLE_02	174,749	0.0/s	0	0	정지삭제

테이블 모니터링 — 실속 이벤트 수·이벤트/s·sink별 lag, 정지/재개/삭제

Deltazium Console
툴로로지
테이블 모니터링
DDL 이력
이벤트
복구
DB 연결
+ CDC 등록
4 running

SCN 지정 재발행

등록 테이블 선택
시작 SCN (이 값부터 재발행)
복구 실행
정합 검증

Iceberg changelog에서 SCN ≥ 시작값을 순서 복원해 복구 토픽으로 재발행하고, recovery-sink(live와 동일 apply 설정)가 타겟에 적용합니다. 경계 중복은 PK upsert 맥동으로 안전합니다.

changelog(S3) 현황 — 복구 가능 범위
새로고침

Iceberg 메타데이터 기준, 파티션(1일)이 남아 있는 구간까지 SCN 재발행 복구가 가능합니다.

changelog.cdc_nh_mix_table_01
86,479 rec · 14 files · 7.0 MB
2026-07-29 ~ 2026-07-30

마지막 커밋 07-30 09:20

2026-07-29 47,840 rec 5 files 3.3 MB
2026-07-30 38,639 rec 9 files 3.8 MB

changelog.cdc_nh_mix_table_02
39,853 rec · 6 files · 3.9 MB
2026-07-30 ~ 2026-07-30

마지막 커밋 07-30 09:20

changelog.src_orders
8 rec · 1 files · 6.2 KB (미분할) ~ (미분할)

마지막 커밋 07-24 11:40

복구 실행 이력
실행 이력이 없습니다.

복구 — changelog(S3) 현황(보존 구간·파티션)·SCN 지정 재발행·go-live 자동 재개·정합 검증

Deltazium Console
툴로로지
테이블 모니터링
DDL 이력
이벤트
복구
DB 연결
+ CDC 등록
4 running

스키마,테이블 필터
전체
INFO
WARN
ERROR
9 events

2026-07-30 09:12:08 CDC.NH_MIX_TABLE_02 REGISTERED CDC 등록·커넥터 배포 (source: orci225 → target: Target97) INFO

2026-07-30 09:07:16 CDC.NH_MIX_TABLE_01 RESUMED apply 재개 — 밀린 분부터 캐치업 INFO

2026-07-30 09:07:13 CDC.NH_MIX_TABLE_01 PAUSED apply 정지 — 캡처·changelog 축적은 계속 INFO

2026-07-30 09:07:06 CDC.NH_MIX_TABLE_01 RESUMED apply 재개 — 밀린 분부터 캐치업 INFO

2026-07-30 09:06:50 CDC.NH_MIX_TABLE_01 PAUSED apply 정지 — 캡처·changelog 축적은 계속 INFO

2026-07-30 09:04:28 CDC.NH_MIX_TABLE_01 RESUMED apply 재개 — 밀린 분부터 캐치업 INFO

2026-07-30 09:03:17 CDC.NH_MIX_TABLE_01 PAUSED apply 정지 — 캡처·changelog 축적은 계속 INFO

2026-07-30 09:02:58 CDC.NH_MIX_TABLE_01 RESUMED apply 재개 — 밀린 분부터 캐치업 INFO

2026-07-30 09:02:49 CDC.NH_MIX_TABLE_01 PAUSED apply 정지 — 캡처·changelog 축적은 계속 INFO

운영 이벤트 타임라인 — 등록·정지·DDL·복구·커넥터 장애 전이(원인 trace 펼침)

Deltazium Console
토폴로지
테이블 모니터링
DDL 이력
이벤트
복구
DB 연결

+ CDC 등록
● 커넥터 1 running
● 3 not running

❏ 캡처 중지 - 전 테이블을 신규 변경 수집 중단 (타킷 apply-changelog에도 새 이벤트가 흐르지 않습니다)

복구 시작

참지: 2026-08-04 15:55 · 원인: io.debezium.DebeziumException: The connector is trying to read change stream starting at OracleOffsetContext [scn=31074668122, txId=01001300ca0d0200, txSeq=41, commit_scn=["31074668165:1:01001300ca0d0200"], lcr_position=null], but this is no longer available on the server. Reconfigure the connector to use a snapshot mode when needed.

▼ 상세 보기 (전체 trace)

```

io.debezium.DebeziumException: The connector is trying to read change stream starting at OracleOffsetContext [scn=31074668122, txId=01001300ca0d0200, txSeq=41, commit_scn=["31074668165:1:01001300ca0d0200"], lcr_position=null], but this is no longer available on the server. Reconfigure the connector to use a snapshot mode when needed.
    at io.debezium.connector.common.BaseSourceTask.validateSchemaHistory(BaseSourceTask.java:161)
    at io.debezium.connector.oracle.OracleConnectorTask.start(OracleConnectorTask.java:154)
    at io.debezium.connector.common.BaseSourceTask.start(BaseSourceTask.java:282)
    at org.apache.kafka.connect.runtime.AbstractWorkerSourceTask.initializeAndStart(AbstractWorkerSourceTask.java:288)
    at org.apache.kafka.connect.runtime.WorkerTask.doStart(WorkerTask.java:191)
    at org.apache.kafka.connect.runtime.WorkerTask.doRun(WorkerTask.java:242)
    at org.apache.kafka.connect.runtime.WorkerTask.run(WorkerTask.java:298)
    at org.apache.kafka.connect.runtime.AbstractWorkerSourceTask.run(AbstractWorkerSourceTask.java:83)
    at org.apache.kafka.connect.runtime.Isolation.Plugins.lambda$switchClassLoader$0(Plugins.java:252)
    at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:572)
    at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:317)
    at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1144)
    at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:629)
    at java.base/java.lang.Thread.run(Thread.java:833)

```

스키마 테이블을 검색

새로고침
재스냅샷

2 tables · ICEBERG LAG은 커밋 주기(60s)만큼 지연이 정상

TABLE	TOPIC	EVENTS	EVENTS/s (실속)	JDBC LAG	ICEBERG LAG	
CDC_NH_MIX_TABLE_01 정지됨	dz.CDC_NH_MIX_TABLE_01	96,949	0.0/s	0	0	재개 삭제
CDC_NH_MIX_TABLE_02 정지됨	dz.CDC_NH_MIX_TABLE_02	50,509	0.0/s	0	0	재개 삭제